

PROYECTO INTEGRADOR

DATA STREAMING

Pipeline end-to-end para monitoreo de pagos confirmados y riesgo por comercio

Apache Kafka + Apache Beam

Curso: Streaming de datos y sus aplicaciones

Programa: Maestría en Inteligencia Artificial - FPUNA

Integrante: Jorge Ismael Barrios Leon

Modalidad: trabajo integrador

Caso de uso: monitoreo operativo de pagos confirmados y señales de alto riesgo por comercio, con manejo explícito de duplicados, desorden temporal, lateness e idempotencia.

1. Problema, usuarios y decisiones habilitadas

El proyecto propone un pipeline de streaming para observar, por comercio y por minuto de tiempo de evento, el monto total confirmado, la cantidad de pagos confirmados y la cantidad de operaciones con fraud_score igual o superior a 0,80. El objetivo operativo es disponer de una señal continua y corregible aun cuando la red entregue eventos fuera de orden, se repitan mensajes o aparezcan registros tardíos. Los usuarios del resultado son equipos de monitoreo, riesgo y auditoría, que pueden identificar concentraciones de pagos, incrementos súbitos de operaciones de alto riesgo y divergencias entre producción, procesamiento y salida.

La elección del caso responde a que los pagos requieren baja latencia sin sacrificar trazabilidad. La solución prioriza corrección técnica, reproducibilidad y observabilidad antes que complejidad innecesaria, en consonancia con la consigna del proyecto integrador (Parra, 2026).

2. Arquitectura end-to-end

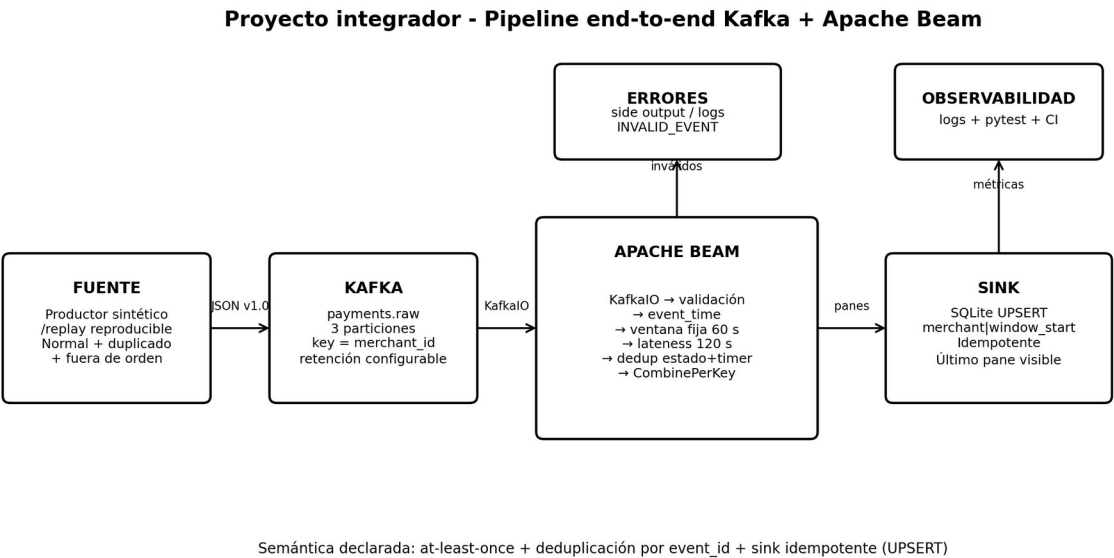


Figura 1. Arquitectura propuesta. Fuente: elaboración propia.

Componente	Tecnología	Responsabilidad	Evidencia
Fuente	Python + confluent-kafka	Generar escenario reproducible con eventos normales, duplicados, fuera de orden, PENDING e inválidos.	Logs PRODUCED y SCENARIO_COMPLETE
Broker	Apache Kafka	Persistir payments.raw en 3 particiones y distribuir por merchant_id.	docker compose ps / tópicos

Procesamiento	Apache Beam	KafkaIO, validación, event_time, ventana, lateness, dedup y CombinePerKey.	Logs DUPLICATE_DROPPED / INVALID_EVENT
Salida	SQLite	UPSERT por merchant_id window_start con último pane visible.	Consulta make results

3. Contrato de eventos y diseño Kafka

3.1 Contrato versionado

Cada evento utiliza `schema_version = 1.0` y contiene `event_id` estable, `key` de negocio, `event_time` del dominio, `event_type` y `payload`. `event_id` se utiliza para deduplicación; `key` coincide con `merchant_id` y se utiliza como clave Kafka; `event_time` conserva el momento real de la operación; `payload` contiene `payment_id`, `merchant_id`, `amount`, `currency`, `status` y `fraud_score`.

Ejemplo JSON

```
{
  "event_id": "evt-001",
  "key": "merchant-301",
  "event_time": "2026-09-30T12:00:05Z",
  "event_type": "payment_status",
  "schema_version": "1.0",
  "payload": {"payment_id": "pay-001", "merchant_id": "merchant-301",
    "amount": 120000, "currency": "PYG",
    "status": "CONFIRMED", "fraud_score": 0.92}
}
```

3.2 Tópicos, clave y particiones

Tópico / salida	Particiones	Clave	Propósito
payments.raw	3	merchant_id	Eventos de entrada; retención y replay del escenario.
payments.invalid	1	-	Reservado para inválidos; la implementación actual los separa y registra en logs.
SQLite payment_metrics	-	merchant_id window_start	Materialización idempotente de la revisión más reciente.

La clave `merchant_id` mantiene afinidad y orden por comercio dentro de una partición. Tres particiones ofrecen paralelismo suficiente para el prototipo. En un despliegue real se monitorizaría skew, porque un comercio dominante podría producir una partición caliente. Kafka permite a los productores usar particionamiento semántico por `key` y a los consumidores controlar su posición en el log, lo que facilita replay (Apache Kafka, 2026).

4. Pipeline con Apache Beam

Beam lee `payments.raw` con `KafkaIO`. `ParseValidateDoFn` valida JSON y contrato; los inválidos se separan mediante `side output` y se registran como `INVALID_EVENT`. Los eventos válidos reciben un `TimestampedValue` basado en `event_time` y se filtran para conservar solo `status = CONFIRMED`. A continuación se aplica una ventana fija de 60 segundos, se clavea por `merchant_id`, se deduplica con

estado por clave/ventana y timer de event time, se agrega con `CombinePerKey` y se adjuntan metadatos de ventana y pane.

El combinador incremental calcula `confirmed_amount`, `confirmed_count` y `high_risk_count`. La salida incluye `pane_timing` y `pane_index` para que el consumidor interprete revisiones. Beam define ventanas sobre los timestamps de los elementos y permite triggers para emitir resultados early y late, equilibrando completitud, latencia y costo (Apache Beam, 2026).

5. Tiempo de evento, ventanas y lateness

Decisión	Configuración	Justificación
Tiempo	event_time del pago	El replay debe conservar la ventana original aun si la llegada cambia.
Ventana	Fija de 60 s [inicio, fin)	Métrica por minuto; comparación no solapada y costo controlado.
Early	Cada 30 s de processing time	Señal operacional rápida y explícitamente provisional.
On-time	Cuando watermark pasa el fin	Mejor estimación de completitud para la ventana.
Late	Por cada evento tardío aceptado	Corrección inmediata del agregado.
Allowed lateness	120 s	Acota estado y admite retrasos moderados.
Acumulación	ACCUMULATING	Cada revisión representa el total conocido y facilita UPSERT.

La política temporal es explícita: un evento fuera de orden conserva su ventana de event_time; un evento tardío puede corregir una ventana mientras el estado permanezca activo; el timer limpia el conjunto de event_id al finalizar window.end + allowed lateness. El watermark no se interpreta como una garantía absoluta de ausencia de futuros eventos, sino como progreso estimado.

6. Duplicados, idempotencia y semántica de entrega

La deduplicación usa SetStateSpec por key y ventana. Cuando reaparece un event_id antes de la expiración, se registra DUPLICATE_DROPPED y no se modifica el agregado. El estado se elimina con un timer de WATERMARK, evitando crecimiento indefinido. Esta frontera temporal se documenta como límite: una repetición posterior a la expiración puede volver a entrar y requerir reproceso controlado.

El sink SQLite utiliza una clave estable merchant_id|window_start y UPSERT. Si se reintenta la misma salida, la fila converge a una única entidad; si llega un pane más nuevo, reemplaza la revisión previa. La semántica declarada es at-least-once en transporte/procesamiento, complementada por deduplicación y sink idempotente. No se afirma exactly-once end-to-end, porque esa garantía exigiría delimitar y demostrar todas las fronteras de efectos externos.

7. Pruebas y evidencia end-to-end

Escenario	Prueba / evidencia	Resultado esperado
Contrato	pytest test_domain	Rechaza campos/fechas inválidas y key inconsistente.
Duplicado	TestPipeline + logs	Mismo event_id por comercio se cuenta una sola vez.
Claves independientes	TestPipeline	Mismo event_id en otro

		comercio no comparte estado.
Fuera de orden	TestStream	Evento mantiene la ventana determinada por event_time.
Idempotencia	SQLite test	Reintentos no generan filas duplicadas; pane reciente prevalece.
End-to-end	make demo	Fuente → Kafka → Beam → SQLite → consumer, con logs adversos.

La evidencia real se genera con make demo y se conserva en evidence/runtime. Deben adjuntarse o enlazarse capturas de docker compose ps, producción, DUPLICATE_DROPPED, INVALID_EVENT, tabla del consumidor y CI en verde. Una ejecución con datos ideales no se considera suficiente; la demostración debe hacer visibles duplicados y desorden temporal.

8. Operación reproducible y observabilidad

El repositorio incluye Dockerfile, docker-compose.yml, Makefile, scripts/e2e_smoke.sh, tests y CI. La secuencia principal es:

make demo

El comando levanta Kafka, crea tópicos, inicia Beam, ejecuta el escenario sintético, consulta la salida y guarda el log de la demostración. La detención y limpieza se realiza con make stop.

Indicadores observables mínimos: eventos producidos, eventos inválidos, duplicados descartados, resultados UPSERT, filas materializadas y estado de CI. Para producción se añadirían métricas de lag, watermark lag, distribución de keys, tamaño de estado, tasa de inválidos y latencia de pane.

9. Supuestos, límites y mejoras

- El proyecto utiliza DirectRunner y un único broker para reproducibilidad local; no representa alta disponibilidad de producción.
- La política de 120 s es una decisión de demostración. En producción debe calibrarse con p95/p99 de event lag.
- La deduplicación tiene horizonte finito; replay históricos extensos requieren una estrategia de identidad persistente.
- SQLite es apropiado para demostración local, no para alta concurrencia. Una evolución natural sería PostgreSQL, BigQuery o un tópico compactado con un servicio de materialización.
- El tópico payments.invalid se crea para evolución posterior; en el núcleo actual los inválidos se registran por side output/logs para simplificar la demostración.
- Como extensión se puede ejecutar Beam sobre Flink, añadir dashboard y medir skew por merchant_id; estas mejoras no sustituyen la corrección del núcleo obligatorio.

10. Integrante y contribución principal

Jorge Ismael Barrios Leon: diseño del caso de uso y arquitectura; contrato versionado; productor sintético; configuración Kafka; pipeline Beam y KafkaIO; política temporal; deduplicación y timer; CombinePerKey; sink SQLite idempotente; pruebas; Docker Compose; evidencia; documentación técnica; preparación de la demostración y defensa.

11. Guion de demostración / video breve

1. Mostrar la arquitectura y explicar el caso de uso (30-45 s).
2. Ejecutar make demo y mostrar Kafka/Beam activos (30 s).
3. Mostrar PRODUCED para eventos normales y el duplicado/fuera de orden (30 s).
4. Mostrar logs DUPLICATE_DROPPED e INVALID_EVENT (30 s).
5. Mostrar make results y explicar la clave merchant_id | window_start y UPSERT (30-45 s).

6. Mostrar pytest/CI en verde y cerrar con límites de la garantía at-least-once + idempotencia (30-45 s).

Referencias

Apache Beam. (2026). Apache Beam Programming Guide.

<https://beam.apache.org/documentation/programming-guide/>

Apache Kafka. (2026). Design. <https://kafka.apache.org/design/>

Parra, R. (2026). Proyecto integrador: Data Streaming. Diseño e implementación de un pipeline end-to-end con Apache Kafka y Apache Beam [Material de clase]. Facultad Politécnica, Universidad Nacional de Asunción.